

AI for the Game of Go: Reinforcement Learning and Classical Search Approaches

Esin Güler

MSKU: Computer Engineering Department

November 4, 2025

Abstract

This project explores artificial intelligence techniques applied to the game of Go on a 5x5 board. The goal is to develop and compare multiple agents, including Random, Minimax, Negamax, Q-Learning, and Deep Q-Learning agents. While classical search algorithms rely on deterministic tree-based evaluations, reinforcement learning agents aim to learn optimal strategies through experience and self-play. We analyze the performance of these agents, discuss their training behaviors, and outline future improvements for scaling to larger board sizes and more complex strategies.

1 Introduction

The goal of this project is to implement AI agents to play the game of Go using different AI strategies. We explore both rule-based methods and reinforcement learning techniques, and in doing so, examine their performance against each other.

2 Project Overview

2.1 Game Environment

Due to outdated libraries, I wrote my own Go environment to simulate basic gameplay, including stone and group capturing, turn passing, scoring, and checking for territory or self-capture moves. This flexible environment allows me to focus more on the AI integration logic. The environment supports interactions with both human and AI agents and tracks the game state for training. A 5x5 Go board is used to make learning easier and faster due to its simpler board evaluation.

2.2 Agents Implemented

The following agents are implemented:

- **Random Agent:** Selects among valid moves randomly.
- **Minimax Agent:** Uses depth-limited search with heuristic evaluation.

- **Negamax Agent:** Variation of minimax for zero-sum games.
- **QLearning Agent:** Reinforcement learning agent with a Q-value table.
- **Deep Q-Learning Agent (DQN):** Neural network approximating Q-values for state-action pairs.

In the next section, we will discuss the implementation details of each agent.

3 Implementation and Training Methodology

3.1 Classical Search Agents: Minimax and Negamax

To provide strong baseline opponents, two classical search-based agents were implemented:

Minimax Agent: The Minimax agent performs a depth-limited search on the game states. After each move, it evaluates all possible upcoming moves up to the depth limit, assuming the opponent also plays optimally. The evaluation uses a heuristic based on stone advantage, number of captured stones, and territory. The maximizing player tries to maximize its score, while the opponent tries to minimize it. Increasing the depth generally improves evaluation accuracy but also increases computation time. In this project, a depth of 4 was used for benchmarking and training reinforcement learning agents.

Negamax Agent: Negamax is a variant of Minimax that is suitable for zero-sum games. Instead of maintaining separate logic for maximizing and minimizing, it recursively evaluates the opponent’s states and negates their values. The formula is:

$$\text{Negamax}(s, d) = -\text{Negamax}(\text{opponent_state}(s), d - 1)$$

where s is the current state and d is the remaining search depth. This simplification makes the code cleaner while producing decisions equivalent to those of Minimax.

Both agents were used as fixed opponents for training and evaluating the learning-based agents.

3.2 Reinforcement Learning Agents

QLearning Agent: The Q-learning agent maintains a Q-table that stores expected rewards for actions taken in specific board states. For each encountered state, the state is added to the Q-table (in our case, implemented as a dictionary), and all possible legal moves are initialized with a default Q-value of 0.0.

During training or evaluation, when the agent encounters a previously seen state, it retrieves the Q-values of all legal moves and decides whether to explore or exploit based on the current epsilon value. After selecting an action and observing the reward, the Q-value for that state-action pair is updated, which estimates the expected future reward of taking action a in state s . The Q-learning update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- s = current state
- a = chosen action
- s' = next state
- r = reward
- α = learning rate
- γ = discount factor

To balance exploitation and exploration, the Q-learning agent uses an epsilon-greedy strategy:

1. A random number between 0 and 1 is generated. If epsilon is higher than this random value, the agent **explores** by selecting a random legal move. This allows the agent to try new actions and discover potentially better strategies.
2. Otherwise, the agent **exploits** its current knowledge by selecting the move with the highest Q-value for the current board state. If multiple moves share the maximum Q-value, one is chosen at random among them.

Keeping a high epsilon value emphasizes exploration, meaning the agent selects moves closer to random. The epsilon value is gradually decayed over time, allowing the agent to increasingly rely on learned strategies rather than random moves.

```
1 if random.random() < epsilon:
2     action = random.choice(valid_moves) # explore
3 else:
4     q_values = self.Q[state]
5     max_q = max(q_values.values())
6     best_moves = [m for m,v in q_values.items() if v == max_q]
7     action = random.choice(best_moves) # exploit
```

The agent was trained using both self-play and matches against the classical agents.

Deep Q-Learning Agent: In contrast, the Deep Q-Learning agent works with Q-values but uses a neural network to approximate $Q(s, a)$ values instead of a Q-table. It stores past experiences, called transitions, in a replay memory. Each state s is passed through the neural network to obtain Q-values for all actions. The agent selects the action with the highest predicted Q-value and updates the network via backpropagation using batches of transitions sampled from the replay memory.

A challenge in training DQN is that the Q-function is constantly moving, and the network is always chasing a target that itself changes after every update. This makes learning unstable. To address this, a separate network called target network is used to compute the TD target, which is updated only periodically. This slows the change in the target, making training more stable and the chasing easier.

The Temporal Difference (TD) target is defined as:

$$y_{TD} = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$$

and the TD loss measures the difference between the predicted Q-value from the policy network (`self.policy_net`) and the TD target:

$$\mathcal{L} = (y_{\text{TD}} - Q_{\text{policy}}(s, a))^2$$

The corresponding implementation in the Go project is:

```

1 # Compute Q-values for current states and next states
2 state_action_values = self.policy_net(state_batch).gather(1,
    action_batch)
3 next_state_values = torch.zeros(self.batch_size, device=device)
4 with torch.no_grad():
5     next_state_values[non_final_mask] = self.target_net(
        non_final_next_states).max(1).values
6
7 # Compute TD target and loss
8 expected_state_action_values = (next_state_values * self.gamma) +
    reward_batch
9 loss = nn.SmoothL1Loss()(state_action_values,
    expected_state_action_values.unsqueeze(1))
10
11 # Backpropagation and network update
12 self.optimizer.zero_grad()
13 loss.backward()
14 torch.nn.utils.clip_grad_value_(self.policy_net.parameters(),
    100)
15 self.optimizer.step()

```

By minimizing this loss over batches of transitions, the network gradually learns to approximate the optimal Q-function, allowing the agent to select effective moves in the 5x5 Go environment.

4 Training Methodology

Training was performed for both Q-Learning and Deep Q-Learning agents, using different parameters and exploration schedules for experimental purposes.

4.1 Q-Learning Agent:

The Q-Learning agent was trained using self-play and matches against other agents. It followed an epsilon-greedy strategy and updated the Q-table using the standard Bellman update rule. The training parameters used are summarized in Table 1.

Rationale: $\alpha = 0.5$ was chosen to balance the speed of learning and stability in Q-value updates. With the value of $\gamma = 0.9$, we are giving importance to long-term rewards while still reacting to immediate outcomes with a little ratio. High initial ϵ values encourage exploration during early self-play, allowing the agent to discover some common random moves. Gradually decaying the ϵ value, shifts the focus towards exploiting learned knowledge, especially when training against stronger opponents like Minimax agent. 10,000 episodes per stage were chosen to provide sufficient experience for the small 5x5 board without excessive computation time.

Table 1: Training parameters for Q-Learning Agent

Stage	Episodes	α	γ	ϵ
1	10,000 (self-play)	0.5	0.9	$0.9 \rightarrow 0.5$
2	10,000 (self-play)	0.5	0.9	$0.5 \rightarrow 0.05$
3	10,000 (vs Random)	0.5	0.9	$0.5 \rightarrow 0.05$
3	10,000 (vs Minimax, depth=4)	0.5	0.9	$0.05 \rightarrow 0.01$
2	10,000 (vs Negamax)	0.5	0.9	$0.5 \rightarrow 0.05$

4.2 Deep Q-Learning Agent

The DQN agent uses a neural network to approximate Q-values, along with replay memory and a target network for stability. Training was performed in three stages with different exploration schedules. Key parameters are summarized in Table 2.

Table 2: Training parameters for Deep Q-Learning Agent

Stage	Episodes	ϵ_{start}	ϵ_{end}	Decay	γ	Batch Size
1	20,000 (self-play)	0.9	0.5	5,000	0.99	128
2	20,000 (self-play)	0.5	0.05	2,500	0.99	128
3	300 (vs Minimax, depth=4)	0.05	0.01	1,000	0.99	128

Rationale: A high initial of ϵ value is chosen for exploring some basic moves in self-play. Reducing ϵ value in later stages encourages exploitation when facing stronger opponents, learning strategies against them. $\gamma = 0.99$ prioritizes long-term rewards more than the immediate ones. Batch size of 128 balances stable gradient estimation with memory and computational efficiency.

This methodology ensured that both Q-Learning and DQN agents progressively learned effective strategies while balancing exploration and exploitation.

5 Implementation Details

5.1 Environment Setup

Go Environment The Go environment was implemented in Python as the `GoEnv` class to simulate gameplay on a 5x5 board. It includes essential features for both learning and evaluation:

- **Board representation:** A NumPy array stores the board with values 1 (black), -1 (white), and 0 (empty).
- **Players and turns:** Tracks the current player and switches automatically after every valid move.
- **Legal moves and self-capture:** The method `get_legal_moves()` returns positions where placing a stone is legal, preventing self-capture.
- **Move execution:** `play_move(x, y)` handles placing a stone, capturing opponent groups, updating the history, and switching turns.

- **Captures and scoring:** Captured stones are tracked, and `score(color)` calculates the total score using Japanese-style scoring with komi.
- **Game termination:** `is_over()` returns `True` if both players pass consecutively or no legal moves remain.

Key internal methods include `get_group()`, `has_liberty()`, `remove_captured_stones()`, and `is_self_capture()`, which implement Go rules like group detection and liberties.

```

1 # Example usage of GoEnv
2 from go_env import GoEnv
3
4 env = GoEnv(size=5) # 5x5 board
5 state = env.reset()
6 done = False
7
8 while not done:
9     action = agent.select_action(state, env)
10    success = env.play_move(*action) # returns False if move
    invalid
11    state = env.board.flatten()      # flattened state for agent
    input
12    done = env.is_over()
13
14 print(env) # prints board state

```

This design allows all agents (Random, Minimax, Negamax, Q-Learning, and DQN) to interact consistently, track the game state, and learn strategies efficiently without focusing on the game-play logic.

It also implements Japanese-style scoring and allows flexible expansion to larger boards or different komi values. On 5x5 boards, I discovered the 2.5 value as a optimum komi value, if i were to increase or decrease that value, I got false calculation of the final score.

Random Agent: The Random Agent serves as a baseline opponent, selecting moves purely at random from the set of legal moves provided by the environment. It does not use any strategy or learning, making it useful for both testing the environment and for initial training of learning agents.

```

1 import random
2
3 class RandomAgent:
4     def select_move(self, goenv):
5         legal_moves = goenv.get_legal_moves()
6         if not legal_moves:
7             return None
8         return random.choice(legal_moves)

```

Minimax Agent: The Minimax agent performs a depth-limited search over possible game states, assuming that the opponent plays optimally. At each node, the agent

evaluates the board using a simple heuristic:

$$\text{Score} = \text{territory} + \text{captured stones} + \text{komi (if WHITE)}$$

The maximizing player aims to maximize this score while the minimizing player attempts to minimize it. Alpha-beta pruning is applied to reduce unnecessary search.

```
1 class MinimaxAgent:
2     def __init__(self, depth=2):
3         self.depth = depth
4
5     def select_move(self, goenv):
6         best_move, best_score = None, -float('inf')
7         ai_color = goenv.get_active_player()
8         for move in goenv.get_legal_moves():
9             temp_env = copy.deepcopy(goenv)
10            temp_env.play_move(*move)
11            score = self.minimax_pruning(temp_env, ai_color,
12                                       depth=self.depth-1,
13                                       alpha=-float('inf'),
14                                       beta=float('inf'),
15                                       maximizing_player=False)
16            if score > best_score:
17                best_score, best_move = score, move
18        return best_move
```

Negamax Agent: Negamax is a variant of Minimax suitable for zero-sum games. Instead of separate logic for maximizing and minimizing players, it recursively evaluates opponent states and negates their scores. This simplifies implementation while producing the almost same optimal decisions.

```
1 class NegamaxAgent:
2     def __init__(self, depth=2):
3         self.depth = depth
4
5     def select_move(self, goenv):
6         best_move, _ = self.negamax(goenv, self.depth, goenv.
7                                     current_player)
8         return best_move
9
10    def negamax(self, goenv, depth, color, alpha=-inf, beta=inf):
11        if depth == 0 or goenv.is_over():
12            return None, self.evaluate(goenv, color)
13        best_score, best_move = -inf, None
14        for move in goenv.get_legal_moves():
15            temp_env = copy.deepcopy(goenv)
16            temp_env.play_move(*move)
17            _, score = self.negamax(temp_env, depth-1, -color, -
18                                   beta, -alpha)
19            score = -score
20            if score > best_score:
21                best_score, best_move = score, move
```

```

20         alpha = max(alpha, score)
21         if alpha >= beta: break
22     return best_move, best_score

```

These classical agents were used as fixed opponents during the training of Q-Learning and DQN agents, providing a benchmark for evaluating learning performance.

Q-Learning Agent: The Q-Learning agent learns optimal moves by maintaining a Q-table, which stores expected rewards for each state-action pair. The board state is represented as a tuple of tuples, making it hashable for dictionary keys.

Epsilon-Greedy Action Selection: At each turn, the agent selects moves based on an epsilon-greedy policy:

- With probability ϵ , it explores by choosing a random legal move.
- With probability $1 - \epsilon$, it exploits the Q-table by choosing the move with the highest Q-value.

Q-Value Update: After taking action a in state s and observing reward r and next state s' , the Q-value is updated using the standard Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- α = learning rate
- γ = discount factor
- a' = possible actions in next state s'

Code Snippet:

```

1  def update(self, state, action, reward, next_state,
2      next_legal_moves):
3      # Initialize missing entries in Q-table
4      if state not in self.Q:
5          self.Q[state] = {action: 0.0}
6      if next_state not in self.Q:
7          self.Q[next_state] = {move: 0.0 for move in
8              next_legal_moves}
9
10     old_q = self.Q[state].get(action, 0.0)
11     max_next_q = max(self.Q[next_state].values()) if self.Q[
12         next_state] else 0.0
13
14     # Standard Q-learning Bellman update rule
15     new_q = old_q + self.alpha * (reward + self.gamma *
16         max_next_q - old_q)
17     self.Q[state][action] = new_q

```


Saving and Loading: The Q-table can be saved or loaded for future use:

```
1 agent.save("q_table.pkl")
2 agent.load("q_table.pkl")
```

The Q-Learning agent was trained using both self-play and against classical agents (Minimax/Negamax/Random), with exploration parameters (ϵ) decayed over time to gradually shift from exploration to exploitation.

Deep Q-Learning : The Deep Q-Learning (DQN) agent approximates the Q-function using a neural network instead of a Q-table. This allows the agent to generalize across unseen board states. The network predicts Q-values for all possible actions in a given state.

Replay Memory: To stabilize training, the DQN agent stores transitions (s, a, r, s') in a replay memory and samples random batches for updates. This helps breaking correlations between sequential experiences and improves learning stability.

Epsilon-Greedy Action Selection: Similar to Q-Learning, the DQN agent selects actions via an epsilon-greedy policy:

- With probability ϵ , it explores by choosing a random legal move.
- With probability $1 - \epsilon$, it exploits the neural network predictions.

Target Network: The agent maintains a target network Q_{target} that is periodically soft-updated from the policy network Q_{policy} . This helps to stabilize the moving target problem in Q-value prediction.

TD Target and Loss: For a batch of transitions, the TD target is calculated as:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$$

The TD loss is computed using the smooth L1 loss:

$$\mathcal{L} = \text{SmoothL1Loss}(Q_{\text{online}}(s, a), y)$$

The network parameters are updated via backpropagation using this loss.

Training Snippet:

```
1 reward = torch.tensor([reward_value], dtype=torch.float32, device
    =device)
2 next_state = None if env.is_over() else torch.tensor(env.board.
    flatten(), dtype=torch.float32, device=device).unsqueeze(0)
3 # Store transition in replay memory
4 agent.memory.push(state, action_tensor, next_state, reward)
5 # Optimize model with a batch of transitions
6 agent.optimize_model()
7 # Soft update target network
8 agent.soft_update_target()
9 state = next_state
```

The DQN agent was trained with self-play and against classical agents, with epsilon decaying over time to shift from exploration to exploitation, and using a batch size of 128 for stable gradient updates.

5.2 Agent Architecture

The learning-based agents are designed to integrate seamlessly with this environment:

- **Q-Learning Agent:** Maintains a Q-table storing state-action values for the 5x5 board. Updates are performed according to the standard Q-learning Bellman equation.
- **Deep Q-Learning Agent (DQN):** Uses a fully connected neural network with two hidden layers to approximate Q-values for all legal actions. It employs a target network and replay memory to stabilize training, and updates the policy network using mini-batch gradient descent.

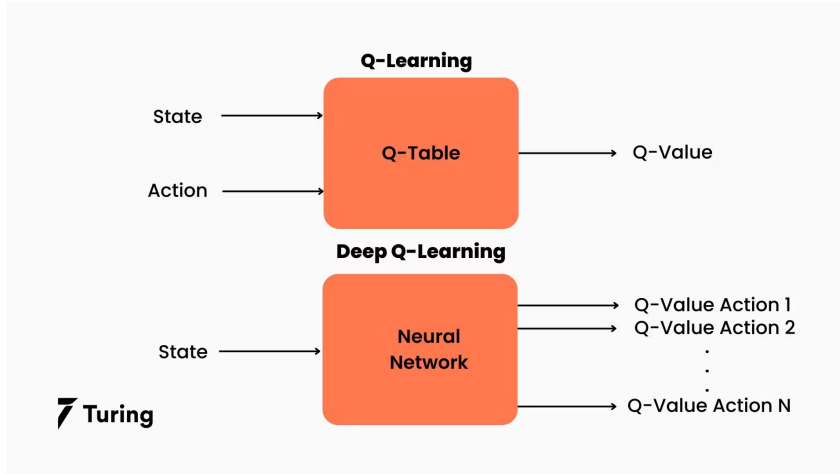


Figure 1: Architecture of DQN and Q-Learning agents.

6 Results and Evaluation

6.1 Benchmarking Methodology

To assess the performance of all implemented agents, a round-robin benchmarking experiment was conducted using the **GoBenchmark** class. Each agent played 10 games against every other agent on a 5x5 Go board. Before evaluation, any available pre-trained models (for Q-Learning or DQN agents) were automatically loaded.

Each match continued until both players passed, and the winner was determined based on the final territory-based score provided by the environment. Performance was measured purely by **win counts** and **win rates** across all matchups.

6.2 Evaluation Criteria

The following metrics were used to summarize the outcomes:

- **Win Count:** Number of games won by each agent.
- **Win Rate:** Percentage of total games won across all opponents.

6.3 Results Summary

After completing all pairwise matches, the win statistics clearly demonstrated that search-based agents (Minimax and Negamax) consistently outperformed reinforcement learning and random agents. The Minimax agent achieved a perfect record, winning every game, while Negamax also performed strongly with a 75% win rate. Q-Learning and DQN agents showed limited success, winning occasionally against Random and each other.

Agent	Win Count	Win Rate (%)
Minimax	40	100.0
Negamax	30	75.0
Random	12	30.0
Q-Learning	10	25.0
DQN	8	20.0

Table 3: Final benchmarking results showing total wins and win rates for each agent after round-robin evaluation.

7 Conclusion

This project successfully implemented and evaluated multiple AI agents for the game of Go on a 5x5 board, integrating both classical search-based methods and reinforcement learning approaches. The agents included Random, Minimax, Negamax, Q-Learning, and Deep Q-Network (DQN) models, all tested within a consistent Go environment framework.

Benchmarking results revealed that the classical search-based agents—particularly Minimax and Negamax achieved the highest performance. Their deterministic evaluation strategies and use of lookahead provided a decisive advantage in the small-board setting, where exhaustive search remains computationally feasible.

In contrast, the reinforcement learning agents (Q-Learning and DQN) exhibited adaptive decision making but required significantly longer training and larger datasets to reach comparable accuracy. These findings emphasize the trade-off between algorithmic precision in handcrafted evaluation functions and the generalization capabilities of learned policies.

Future work could explore scaling to larger board sizes, extending the training duration for learning-based agents, and incorporating more advanced deep reinforcement learning architectures such as AlphaZero-style Monte Carlo Tree Search (MCTS). Overall, this project offered valuable insights into how classical and learning-based AI paradigms approach strategic decision-making in Go.

8 References

- Datacamp. (n.d.). *Introduction to Q-Learning: A Beginner's Tutorial*. Retrieved from <https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial>

- PyTorch. (n.d.). *Reinforcement Learning (DQN) Tutorial*. Retrieved from https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html
- Turing.com. (n.d.). *How Neural Networks Are Used in Deep Q-Learning*. Retrieved from <https://www.turing.com/kb/how-are-neural-networks-used-in-deep-q-learning>
- Sentdex. (2018). *Deep Q-Learning Tutorial — Neural Network for Reinforcement Learning* [YouTube Video]. Retrieved from <https://youtu.be/1-hh51ncgDI>
- Python Libraries Used: NumPy, PyTorch, Matplotlib.